WildTrace 使用说明书（赛题交付件 3 · 文档部分）

> 版本：1.1（2026-10-22，对应 master tag `v2.4`；1.0 为 2026-09-30 / `a023018`）。配套演示视频按 `docs/deliverables/VIDEO_SCRIPT.md` 的分镜录制。分工：**本文件是交付件三正文**（依赖环境 + 主命令 + 常见问题 + 交付件索引）；`USER_GUIDE.md` 是同一交付件的命令行补充页（分步 walkthrough、安全边界与工作台 说明），两者不重复计数，冲突时以本文件为准。

1. 依赖环境

项	要求	说明
操作系统	Windows 10+ / Linux	两条路径均已验证；路径含空格可用（已做引用处理）
Python	3.10+	核心扫描/报告零第三方运行时依赖（`pip install -e . --no-deps` 即装即用）
Git	可执行	知识库同步与 profile 来源校验需要；纯扫描不需要
Node.js（可选）	18+ 与 npm	仅工作台前端构建需要（`Start-Workbench.ps1` 会执行 `npm ci` + `npm run build`）；纯扫描、报告、PoC 验证均不需要
WSL（可选）	Ubuntu 24.04 + LLVM/Clang	Clang AST 引擎、Joern（`/opt/joern`）、运行时验证矩阵需要
LLM 密钥（可选）	`OSLOVELACE_API_KEY` 环境变量	仅 `ohvul.llm_review` / `ohvul.llm_poc` 选配功能；默认扫描全程离线
C 编译器（可选）	`zig cc`（`pip install ziglang`）或 `WILDTTRACE_CC`	仅 PoC 验证（`ohvul.poc_verify`）需要；不装则验证步骤跳过，扫描不受影响

安装与自检：

```
# ① 核心功能（扫描 / 报告 / 影响矩阵 / PoC 闸门）：不装任何第三方包
python -m pip install -e . --no-deps
python -m unittest discover -s tests -q        # 当前用例 1,248 项（发现计数）
python -m ohvul.toolchain --output preflight.json   # 工具链预检（演示第一步）

# ② 工作台（可选）：Web 依赖走 extras，另需 Node 18+ 构建前端
python -m pip install -e ".[workbench]"
.\Start-Workbench.ps1                          # 内部会装 extras、npm ci、npm run build
```

> 只装 ① 时 `python -m ohvul.workbench.launch` 会报 `No module named 'fastapi'` —— 工作台是可选组件，不是缺陷；`.[workbench]` 会拉 fastapi/uvicorn/httpx 三个包。

`preflight.json` 中 `ok=true` 表示 WSL/Clang/clang-tidy 通道可用；任一 required 项失败时按 "预检失败不继续" 纪律处理（这是演示的一个环节，不是缺陷）。

2. 主命令速查

2.1 自动化分析管线（autopilot v3）—— 主入口

```
python -m ohvul.autopilot .\authorized-source --output .\out [--min-confidence 60] [--joern]
```

- 默认 **10 层**：P1–P3（CJ FFI 面三族，仅当树含 `/cj/` 路径时运行）+ P4–P9（通用 C/C++ 六族）+ 跨函数 UAF/双重释放（位置别名 + 局部变量追踪）
- `--joern` 增加**第 11 层**（Joern CPG，需 WSL `/opt/joern`；不可用时警告降级，不中断）
- 产出：`findings.json`（含逐条 0-100 置信度与因子分解、PoC 模板）+ `findings.md`（人读报告）
- 口径：**lead ≠ 漏洞**——置信度是排序优先级，不是准确率保证

2.2 影响矩阵与交付件组装

```
python -m ohvul.impact_matrix --cases cases --output docs/deliverables/IMPACT_MATRIX
python -m ohvul.vuln_report --cases cases --output docs/deliverables/VULN_REPORT.md --force
python -m ohvul.competition_report --cases cases --output out\report.json --markdown out\report.md
```

影响矩阵按 CVE 联接知识库（`kb/third_party_targets`，cvelistV5）做证据分级判定； 两个报告生成器均不发明数字（？= 待定，confirmed 只投影显式字段）。

2.3 离线证据链管线（Phase 1 纪律层）

```
python -m ohvul.cli .\authorized-source --output findings.json --checkpoint ck.json
python -m ohvul.candidate_triage --result findings.json --output triage.json
python -m ohvul.verify findings.json
```

候选一律待人工复核；`input_error` 不计为干净扫描。

2.4 选配：LLM PoC 生成（已加固）

```
# 默认通道 GLM；finding 路径限制在 source_root 内，输出过危险 API 屏蔽
python -m ohvul.llm_poc .\out\findings.json .\authorized-source --output llm_pocs --min-confidence 70

# 换 DeepSeek（OpenAI Chat-Completions 通道，端点在白名单内）
python -m ohvul.llm_poc .\out\findings.json .\authorized-source --output llm_pocs `
  --base-url https://api.deepseek.com/v1 --model deepseek-chat
```

产出 `llm_pocs/*.c` + `llm_poc_summary.json`（每条草稿对应的 finding 位置，验证步骤 要用它判定"是否抄了目标函数"）。密钥只经 `OSLOVELACE_API_KEY` 运行时注入。

2.5 PoC 四道闸门：编译 → 运行 → 帧归因（2026-10-21 起）

生成完必须过闸门，否则草稿不构成证据。**不指定 `--link` 时链接集自动推断：** 编译 → 读链接器的 undefined symbol → 把同目录定义该符号的源文件加进链接集 → 重试， 上限 3 轮。

```
# 源码级必现（空指针/格式化串）：普通构建即可判定
python -m ohvul.poc_verify llm_pocs --source .\authorized-source `
  --findings .\out\findings.json --output .\verify

# UAF / 溢出（静默腐蚀类）：必须带内存检查器，否则"没崩"只判 undetermined
python -m ohvul.poc_verify llm_pocs --source .\authorized-source `
  --findings .\out\findings.json --output .\verify `
  --guarded-heap tools\harness          # Windows：预处理器改名 + 隔离页
  # Linux 预编译 .so 场景改用 LD_PRELOAD：tools/harness/linux/guarded_heap.c

# 有外部取帧结果时（poc_real 或 run_real.sh 产出）再喂回去，区分"崩在库内"与"崩在这一行"
python -m ohvul.poc_verify llm_pocs --source .\authorized-source `
  --findings .\out\findings.json --output .\verify --frames frames.json
```

判读四档，看 `poc_verification.json` 的 `verdict`：

verdict	含义	处置
`confirmed`	链真库 + 崩溃 + 帧与 finding 函数吻合	可入报告
`wrong_site`	崩在库内，但帧在无关函数（ABI 猜错等）	改 PoC 重跑，不可当 confirmed
`refuted`	链真库 + 跑完 + 库未崩（且该模式有内存检查器覆盖）	该路径不可达
`undetermined`	未编译/未运行/副本崩溃/无检查器/未链库	不下结论

真库（预编译 OHOS `.so`）走 `ohvul.poc_real` 或 `tools/harness/ohos/run_real.sh <OH_ROOT> <BUILD_OUTPUT> <client.c> <lib.so> <evidence>`： WSL + musl 加载器 + gdb 两轮 + `llvm-addr2line -i`，输出逐帧 JSON。

2.6 知识库与基准

```
python -m ohvul.kb_sync build --kb-dir <dir>     # CVE 知识库构建/校验
python -m ohvul.benchmark cases.json             # 配对真假基准（builtin vs function_rules 可对比）
python tools/kb_gap_audit.py <component-repo> --kb <kb.json>   # 回移缺口四信号审计
```

3. 扫描覆盖率与粒度（对应评价备注 7）

层	覆盖面	粒度
autopilot 规则层（P1–P9 + 跨函数）	树内全部 `.c`/`.cpp` 文件（无白名单）	函数级（注释/字符串掩码后的结构切分）
Joern 层（`--joern`）	同上（整树 CPG）	路径敏感 CPG 查询
Clang AST 层	按组件 profile 声明的文件清单	编译器事实级（含精确值/区间标量流）
覆盖率口径	`findings.json` 的 `surface` 字段如实记录扫描面（文件数/行数估计/CJ 命中）；声明外文件不计为已分析	

实测示例见 `docs/deliverables/PERFORMANCE.md`（dsoftbus 1,737 文件 / 681,529 行、 651.67 s、3,033 条 lead；CVE 富集 6 组件 2,339,964 行 / 8,750.8 s；耗时与内存数据随附）。

4. 常见问题

- **扫描输出目录**：必须在被扫描树之外；已存在的报告默认拒绝覆盖（`--force` 显式覆盖）。
- **WSL 不可用**：Clang/Joern 相关功能降级或提示，不影响规则层扫描。
- **行尾**：仓库已配 `.gitattributes`（`* text=auto`，`*.sh`/`*.sc` 强制 LF），Windows 检出可直接用。
- **密钥安全**：密钥仅运行时读取，不落盘不入日志；CI 含密钥形态扫描。
- **没有 ASan**：MinGW 缺 ASan 运行时（`undefined symbol: __asan_report_load8`）， UAF/溢出类草稿"没崩"不能算证伪——加 `--guarded-heap tools\harness` 让隔离页 自己把 UAF 变成故障，否则判定固定为 `undetermined`。
- **判成 `wrong_site`**：说明崩在库里但不是被告知的那一行（典型是 ABI/结构体猜错）。 按符号化帧回到源码核对参数形态再改一版，不要直接把退出码 139 当成功。
- **zig 首次构建报 `SubCompilationFailed`**：mingw CRT 重建的瞬态错误，工具会按 原命令自动重试一次，不消耗模型重写预算。

5. 交付件索引

`docs/deliverables/`：TECHNICAL_SPEC（技术说明书）、VULN_REPORT（漏洞分析报告，`.md` + `.pdf`）、IMPACT_MATRIX（影响矩阵）、PERFORMANCE（性能说明）、VIDEO_SCRIPT（演示分镜）、USAGE（本文件，交付件三正文）、USER_GUIDE（同交付件的命令行补充页）。状态台账在 `docs/status/`，审计与实验记录在 `docs/analysis/`，PoC 验证 harness 在 `tools/harness/`（Windows 守卫堆 / Linux `LD_PRELOAD` / OHOS 真库 `run_real.sh`）。

交付件导出（Markdown → PDF，需要 `pip install markdown` 与本机 Chrome/Edge）：

```
python tools/export_pdf.py docs/deliverables/VULN_REPORT.md --output docs/deliverables/VULN_REPORT.pdf
```

6. 本地工作台（2026-10 增补）

```
.\Start-Workbench.ps1                # 或：python -m ohvul.workbench.launch
# 浏览器打开 http://127.0.0.1:8765
```

- 总览页：案例证据完整度指标（计数无关断言）、扫描/任务状态
- 案例对话框：源码位置 / PoC 与附件 / 版本影响三标签，证据链字段直读 case.json
- 候选审查页与报告中心：发现队列、报告预览与导出；双引擎（P6 规则层 + Joern CPG 选配）同一界面驱动
- 数据目录默认 `.cache/workbench`，可用 `--data-dir` 指定

两处前置条件（2026-10-22 补，照做否则会卡住）

- **源码根白名单**：工作台只放行仓库内 `bench/` 与 `C:/Projects/OpenHarmony`、`C:/Projects/OpenHarmony-complete` 三处。在"扫描任务"页填其他目录会直接返回 `Path is outside configured roots` —— 先在"设置"页登记该源码根，再启动扫描。
- **"运行验证"页的登记范围**：首版只接受 `C:/Projects/OpenHarmony/third_party/cJSON` 一处源码根。其预检同时校验四件事：WSL（`wsl -d Ubuntu`）内 `/usr/bin/bwrap` 与 `/home/airgap/llvm/bin/clang` 在位；该目录 `git rev-parse HEAD` 等于 `cases/cjson_merge_alias/case.json` 的 `expected_commit`（`388cfc96925e4fb41a30735b270736626d26615d`）；工作树干净；`cJSON.c` 与 `cJSON_Utils.c` 的 SHA-256 等于记录值。任一不满足即返回 `ready=false` 并给出原因、不进入运行队列——这是"预检不通过不执行"的落点，不是功能缺陷。

准备这棵源码树（用已有的 third_party_cJSON 克隆即可，无需拉整仓）：

`bash git -C <third_party_cJSON 克隆目录> worktree add --detach \ C:/Projects/OpenHarmony/third_party/cJSON 388cfc96925e4fb41a30735b270736626d26615d`

7. 验证 harness 一键化（2026-10 增补）

`tools/validation_cases/`（17 个 .c/.sh，含 20+ 验证客户端）：

runner	用途
`run_malformed.sh`	任意真库 malformed 客户端的编译-加载-执行-封存（udmf/ipc/kv/hiappevent 等，exit 139 判读）
`run_ipc_normal.sh`	IPC 正常路径 round-trip（对照组）

runner	用途
`run_bt_dereg.sh`	bluetooth 服务端 DeregisterServerObserver PoC：构建到帧归因一键完成（符号化 `bluetooth_socket_server.cpp:54`）

其余单案客户端（`*_null_*.c`、`bluetooth_neg_advert.c` 等）均以 `run_malformed.sh <root> <out> <evid> <client> <lib>` 统一驱动，证据哈希 前后封存。LLM 侧：`tools/llm_review_run.py`（plan/run/verify）与 `tools/llm_cross_compare.py`。